

«УНИВЕРСИТЕТ ИННОПОЛИС»

Образовательная программа «Математические основы искусственного
интеллекта»

ПРОЕКТНАЯ РАБОТА

по дисциплине «Современные численные методы
оптимизации в обучении»

**«Сравнение работы обратного
распространения ошибки для разных
вариантов функции потерь (MSE и
Cross-Entropy)»**

Выполнили студентки:

Хадралинова Полина Александровна

Чиркова Анна Юрьевна

Проверил:

к.ф.-м.н., доцент

Стонякин Фёдор Сергеевич

Содержание

1	Введение	2
2	Математические основы выбора функции потерь	4
2.1	Проблема насыщения и затухания градиента (Sigmoid + MSE)	4
2.2	Преимущества перекрестной энтропии	5
2.3	Обобщение на многоклассовую классификацию (Softmax) .	6
2.3.1	Анализ «паралича» градиента	7
2.3.2	Математическое согласование Softmax и Cross-Entropy	8
3	Методика эксперимента	10
3.1	Описание набора данных	10
3.2	Метрики качества	10
3.3	Подготовка данных и предобработка	10
3.4	Архитектура нейронной сети	11
3.5	Параметры экспериментальной настройки	11
4	Программная реализация алгоритмов	12
4.1	Математическое ядро модели	12
5	Отчет о выполненных экспериментах	14
5.1	Динамика сходимости потерь и точности	14
5.2	Анализ финальных метрик качества	16
5.3	Визуализация структуры ошибок (Confusion Matrix)	17
6	Заключение	19
	Замечание о влиянии инициализации и отсутствии регуляризации	19
	Приложение А: Полный исходный код программной реализации	21

1 Введение

Современное машинное обучение неразрывно связано с решением сложных задач математической оптимизации. В основе обучения искусственных нейронных сетей лежит принцип минимизации эмпирического риска (функции потерь). Процесс подбора оптимальных весов модели осуществляется с помощью градиентных методов, где ключевую роль играет алгоритм обратного распространения ошибки (backpropagation).

Успех и скорость сходимости алгоритма оптимизации критически зависят от выбранной функции потерь. В данной работе исследуется влияние функции потерь на динамику обучения нейронной сети. Основной целью проекта является математическое и практическое сравнение двух классических функций: MSE (Mean Squared Error) и Cross-Entropy (перекрестная энтропия).

В лекционном курсе эффект затухания градиентов (вызванный попаданием функции активации в зону насыщения) при использовании MSE рассматривался на примере бинарной классификации с применением функции активации «сигмоида». Однако для практического исследования мы выбрали классическую задачу многоклассовой классификации — распознавание рукописных цифр из набора данных MNIST.

Использование независимых сигмоид для данной задачи математически некорректно, так как классы являются взаимоисключающими. Поэтому архитектура нашей модели модифицирована: на выходном слое применяется функция **Softmax**, которая преобразует выходы сети в корректное распределение вероятностей.

В рамках проекта мы ставим перед собой следующие задачи:

1. Математически обосновать, почему связка *Softmax* + *MSE* страдает от проблемы затухания градиента при сильной уверенности модели в неверном ответе.
2. Показать, как функция *Cross-Entropy* решает данную проблему за счет логарифмического сглаживания.
3. Вручную на базе библиотеки NumPy реализовать алгоритмы прямого и обратного распространения ошибки для полносвязной ней-

ронной сети.

4. Провести численный эксперимент на датасете MNIST и сравнить модели по графикам сходимости и расширенным метрикам качества.

2 Математические основы выбора функции потерь

Фундаментальная цель градиентной оптимизации — минимизация функции потерь L путем итеративного обновления весовых коэффициентов w модели. В базовом алгоритме градиентного спуска (Stochastic Gradient Descent) правило обновления на шаге t имеет вид:

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w} \quad (1)$$

где η — скорость обучения (learning rate), а $\frac{\partial L}{\partial w}$ — градиент функции потерь по конкретному весу.

Из данной формулы очевидно: если градиент $\frac{\partial L}{\partial w}$ стремится к нулю, то $w^{(t+1)} \approx w^{(t)}$. Процесс оптимизации останавливается, и сеть перестает обучаться, даже если текущая ошибка велика.

Рассмотрим математические причины, по которым среднеквадратичная ошибка демонстрирует низкую эффективность в задачах классификации, обучаемых с помощью градиентного спуска.

2.1 Проблема насыщения и затухания градиента (Sigmoid + MSE)

Пусть имеется один нейрон, выход которого вычисляется с использованием сигмоидной функции активации:

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{где } z = wx + b \quad (2)$$

При использовании функции потерь MSE стоимость ошибки C определяется как:

$$C = \frac{(a - y)^2}{2} \quad (3)$$

где $y \in \{0, 1\}$ — истинная метка класса.

В процессе обучения алгоритм градиентного спуска обновляет вес w пропорционально частной производной функции потерь $\frac{\partial C}{\partial w}$. Применяя

цепное правило дифференцирования, получаем градиент по весу w :

$$\frac{\partial C}{\partial w} = \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = (a - y) \cdot \sigma'(z) \cdot x \quad (4)$$

Производная сигмоидной функции выражается через сам выход нейрона:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a). \quad (5)$$

Тогда градиент принимает вид:

$$\frac{\partial C}{\partial w} = (a - y) \cdot a(1 - a) \cdot x \quad (6)$$

Множитель $a(1 - a)$ достигает максимума 0.25 при $a = 0.5$ (нейрон «не уверен») и строго стремится к нулю, когда выход приближается к 0 или 1 (нейрон «насыщен»). Рассмотрим ситуацию сильной ошибки:

- Если истинная метка $y = 1$, а модель ошибочно выдаёт $a \approx 0$, то ошибка $(a - y) \approx -1$ велика, однако $a(1 - a) \approx 0 \cdot 1 = 0$. Произведение даёт почти нулевой градиент.
- Если $y = 0$, а модель предсказывает $a \approx 1$, то $(a - y) \approx 1$, но $a(1 - a) \approx 1 \cdot 0 = 0$ с тем же результатом.

Таким образом, когда сеть глубоко заблуждается и при этом «излишне уверена» в неверном ответе, градиент (6) оказывается ничтожно малым. Веса практически не обновляются, и модель застывает в ошибочном состоянии.

2.2 Преимущества перекрестной энтропии

Для решения проблемы затухающего градиента необходимо было подобрать такую функцию потерь, которая математически компенсировала бы сплющивание производной функции активации. В случае использования сигмоиды для бинарной классификации идеальным решением стала функция Binary Cross-Entropy:

$$C = -(y \ln(a) + (1 - y) \ln(1 - a)) \quad (7)$$

Рассмотрим, как меняется частная производная $\frac{\partial C}{\partial a}$ в этом случае:

$$\frac{\partial C}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a} = \frac{a-y}{a(1-a)} \quad (8)$$

Так как производная сигмоиды $\frac{\partial a}{\partial z} = a(1-a)$, при применении цепного правила происходит математическое сокращение:

$$\frac{\partial C}{\partial w} = \frac{a-y}{a(1-a)} \cdot a(1-a) \cdot x = (a-y) \cdot x \quad (9)$$

В итоговой формуле (9) исчезла производная функции активации. Величина градиента теперь зависит только от величины ошибки $(a - y)$. Чем сильнее предсказание a отличается от истинного значения y , тем больше будет шаг обновления весов. Проблема затухания градиента полностью решена.

2.3 Обобщение на многоклассовую классификацию (Softmax)

Для решения задачи распознавания 10 цифр MNIST на выходном слое используется функция **Softmax**. В отличие от сигмоиды, она преобразует вектор логитов $\mathbf{z}$ в распределение вероятностей $\mathbf{a}$, где сумма всех элементов равна единице:

$$a_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}, \quad i = 1, \dots, 10 \quad (10)$$

Ключевая особенность Softmax заключается в том, что каждый выход a_i зависит от всех входных логитов $z_1, \dots, z_{10}$. Это обуславливает необходимость использования матрицы Якоби при дифференцировании. Частные производные Softmax имеют вид:

$$\frac{\partial a_k}{\partial z_i} = \begin{cases} a_i(1 - a_i), & \text{если } k = i \\ -a_i a_k, & \text{если } k \neq i \end{cases} \quad (11)$$

Рассмотрим вывод градиента функции потерь **MSE** $L = \frac{1}{2} \sum_k (a_k -$

$y_k)^2$ по логиту z_i . Согласно цепному правилу, нам необходимо просуммировать влияние логита z_i на каждый из 10 выходов:

$$\frac{\partial L}{\partial z_i} = \sum_{k=1}^{10} \frac{\partial L}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_i} \quad (12)$$

Раскроем сумму в уравнении (12), выделив член, где $k = i$, и остальные члены, где $k \neq i$:

$$\frac{\partial L}{\partial z_i} = \underbrace{(a_i - y_i) \cdot a_i(1 - a_i)}_{\text{влияние на свой выход}} + \sum_{k \neq i} \underbrace{(a_k - y_k) \cdot (-a_i a_k)}_{\text{влияние на чужие выходы}} \quad (13)$$

Вынесем общий множитель a_i за скобку и упростим выражение:

$$\begin{aligned} \frac{\partial L}{\partial z_i} &= a_i \left[(a_i - y_i)(1 - a_i) - \sum_{k \neq i} a_k(a_k - y_k) \right] \\ &= a_i \left[(a_i - y_i) - a_i(a_i - y_i) - \sum_{k \neq i} a_k(a_k - y_k) \right] \\ &= a_i \left[(a_i - y_i) - \sum_{k=1}^{10} a_k(a_k - y_k) \right] \end{aligned} \quad (14)$$

2.3.1 Анализ «паралича» градиента

Формула (14) наглядно объясняет, почему модель с MSE застревает на «плато». Рассмотрим критическую ситуацию: модель на вход получила цифру «5» ($y_5 = 1$, остальные $y_k = 0$), но ошибочно и очень уверенно предсказала цифру «3» ($a_3 \approx 1$, следовательно $a_5 \approx 0$).

Как будет обновляться логит правильного ответа z_5 ?

- В формуле (14) перед всей скобкой стоит множитель a_5 .
- Так как наша модель почти не верит в правильный ответ ($a_5 \approx 0.0001$), то всё значение градиента умножается на это мизерное число.
- В результате, даже при колоссальной ошибке, вес нейрона практически не получает сигнала к изменению.

Как будет обновляться логит ошибочного ответа z_3 ?

- Здесь $a_3 \approx 1$. Внутри скобки первое слагаемое $(a_3 - y_3) = (1 - 0) = 1$.
- Второе слагаемое (сумма) будет также стремиться к 1, так как в ней доминирует член $a_3(a_3 - y_3) \approx 1 \cdot 1 = 1$.
- Происходит взаимное вычитание в скобках: $1 - 1 = 0$.

Вывод: В обоих случаях градиент стремится к нулю. Модель оказывается в состоянии «самоуверенного заблуждения»: она совершает огромную ошибку, но её математическая структура (Softmax + MSE) подавляет попытки эту ошибку исправить.

2.3.2 Математическое согласование Softmax и Cross-Entropy

Для доказательства превосходства **Categorical Cross-Entropy (CCE)**, приведем аналитический вывод её градиента в связке с Softmax. Функция потерь имеет вид:

$$L_{CE} = - \sum_{k=1}^{10} y_k \ln(a_k) \quad (15)$$

Найдем производную по логиту z_i , применяя цепное правило и Якобиан (11):

$$\frac{\partial L_{CE}}{\partial z_i} = \sum_{k=1}^{10} \frac{\partial L_{CE}}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_i} = \sum_{k=1}^{10} \left(-\frac{y_k}{a_k} \right) \frac{\partial a_k}{\partial z_i} \quad (16)$$

Разобьем сумму (16) на два случая ($k = i$ и $k \neq i$):

$$\frac{\partial L_{CE}}{\partial z_i} = -\frac{y_i}{a_i} \cdot a_i(1 - a_i) - \sum_{k \neq i} \frac{y_k}{a_k} \cdot (-a_i a_k) \quad (17)$$

Сократим дроби:

$$\frac{\partial L_{CE}}{\partial z_i} = -y_i(1 - a_i) + \sum_{k \neq i} y_k a_i \quad (18)$$

Раскроем скобки и вынесем a_i из суммы:

$$\frac{\partial L_{CE}}{\partial z_i} = -y_i + y_i a_i + a_i \sum_{k \neq i} y_k = -y_i + a_i \left(y_i + \sum_{k \neq i} y_k \right) \quad (19)$$

Так как y — это One-Hot вектор (только одна единица, остальные нули), то сумма всех его компонентов строго равна единице: $\sum_{k=1}^{10} y_k = 1$. Подставляя это, получаем итоговый элегантный результат:

$$\frac{\partial L_{CE}}{\partial z_i} = -y_i + a_i(1) = a_i - y_i \quad (20)$$

Вывод: В процессе дифференцирования логарифм из Cross-Entropy и экспонента из Softmax математически "уничтожают" нелинейности друг друга. Из итоговой формулы полностью исчез множитель активации. Градиент становится строго линейным и зависит исключительно от величины допущенной ошибки $(a_i - y_i)$. Это гарантирует, что оптимизатор никогда не застрянет на плато, что полностью подтверждается результатами нашего практического эксперимента.

3 Методика эксперимента

3.1 Описание набора данных

Для численного сравнения функций потерь используется датасет MNIST. Набор содержит 70,000 изображений рукописных цифр:

- Размер каждого изображения: 28×28 пикселей
- Количество входных признаков: 784 (яркость каждого пикселя)
- Количество классов: 10 (цифры от 0 до 9)

3.2 Метрики качества

Для оценки эффективности моделей мы используем следующие метрики на основе Матрицы ошибок (Confusion Matrix):

1. **Accuracy (Точность)** — доля правильных ответов.
2. **Confusion Matrix** — таблица 10×10 , показывающая структурные ошибки сети.
3. **Macro F1-Score** — гармоническое среднее между точностью и полнотой. Данная метрика особенно важна в нашем исследовании, так как позволяет выявить ситуации, когда модель "угадывает" только определенные классы.

3.3 Подготовка данных и предобработка

Для корректной работы градиентных методов данные были подвергнуты процедуре нормализации:

1. **Нормализация яркости:** Исходные значения пикселей $[0, 255]$ были масштабированы в диапазон $[0, 1]$ путем деления на 255.0. Это критически важно для предотвращения численного переполнения при вычислении экспонент в Softmax.
2. **One-Hot Encoding:** Целевые метки цифр (от 0 до 9) были преобразованы в бинарные векторы размерности 10.

3. **Разделение выборки:** Данные были разделены на обучающую (80%) и тестовую (20%) части. Обучение проводится на тренировочных данных, а итоговая оценка — на тестовых.

3.4 Архитектура нейронной сети

В работе реализована трехслойная полносвязная нейронная сеть (Multilayer Perceptron) со следующей структурой:

- **Входной слой:** 784 нейрона.
- **Скрытый слой:** 64 нейрона с функцией активации **ReLU** ($f(z) = \max(0, z)$). Использование ReLU позволяет избежать затухания градиентов внутри скрытых слоев, фокусируя исследование именно на проблемах выходного слоя.
- **Выходной слой:** 10 нейронов с функцией активации **Softmax**.

3.5 Параметры экспериментальной настройки

Обучение обеих моделей (CCE и MSE) запускалось с идентичными начальными весами для обеспечения чистоты сравнения. Были выбраны следующие гиперпараметры:

- **Количество эпох:** 90 (увеличенный горизонт позволяет наглядно увидеть поведение MSE на плато).
- **Размер батча:** 128.
- **Скорость обучения (Learning Rate):** 0.015.

4 Программная реализация алгоритмов

Согласно требованиям учебного курса, алгоритмы оптимизации и обратного распространения ошибки были реализованы полностью вручную на языке Python с использованием библиотеки *NumPy*.

4.1 Математическое ядро модели

В коде ниже представлены функции прямого и обратного прохода. Особое внимание уделено корректному расчету градиентов $dZ2$ (ошибки на выходном слое) для разных функций потерь в соответствии с формулами, выведенными в Главе 1.

```
def forward(self, X):
    self.Z1 = np.dot(X, self.W1) + self.b1
    self.A1 = np.maximum(0, self.Z1) # ReLU activation

    self.Z2 = np.dot(self.A1, self.W2) + self.b2
    # Softmax activation
    expZ = np.exp(self.Z2 - np.max(self.Z2, axis=1, keepdims=True))
    self.A2 = expZ / np.sum(expZ, axis=1, keepdims=True)
    return self.A2

def backward(self, X, Y_true, lr, loss_type):
    m = X.shape[0]

    if loss_type == 'ce':
        dZ2 = self.A2 - Y_true # Perfect Gradient
    else: # 'mse'
        # Gradient MSE + SoftMax
        diff = self.A2 - Y_true
        dZ2 = self.A2 * diff - self.A2 * np.sum(self.A2 *
            diff, axis=1, keepdims=True)

    dW2 = (1/m) * np.dot(self.A1.T, dZ2)
    db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)

    dA1 = np.dot(dZ2, self.W2.T)
    dZ1 = dA1 * (self.Z1 > 0) # derivative ReLU
```

```

dW1 = (1/m) * np.dot(X.T, dZ1)
db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)

# update the weights
self.W2 -= lr * dW2
self.b2 -= lr * db2
self.W1 -= lr * dW1
self.b1 -= lr * db1

```

Для обучения моделей был реализован алгоритм **Mini-batch Stochastic Gradient Descent**. Данный метод вносит в процесс оптимизации элемент стохастичности, что помогает алгоритму выходить из неглубоких локальных минимумов. Перед каждой эпохой данные подвергаются процедуре случайной перестановки, после чего происходит итеративное обновление весов по пачкам данных размером 128 образцов.

```

# Initialization of experimental parameters
epochs = 90
batch_size = 128
lr = 1.5e-2

for i in range(epochs):
    # Shuffling the sample to ensure stochasticity
    permutation = np.random.permutation(m_train)
    X_shuffled = X_train[permutation]
    Y_shuffled = Y_train_oh[permutation]

    # Iterations
    for j in range(0, m_train, batch_size):
        X_batch = X_shuffled[j:j+batch_size]
        Y_batch = Y_shuffled[j:j+batch_size]

        # Parallel training of two identical networks (CCE
        # and MSE)
        nn_ce.forward(X_batch)
        nn_ce.backward(X_batch, Y_batch, lr, 'ce')

        nn_mse.forward(X_batch)
        nn_mse.backward(X_batch, Y_batch, lr, 'mse')

```

Полный исходный код проекта с циклами обучения, сбором метрик и построением графиков приведен в **Приложении А**.

5 Отчет о выполненных экспериментах

В ходе эксперимента были получены данные, подтверждающие критическое преимущество логарифмических функций потерь перед квадратичными в задачах классификации.

5.1 Динамика сходимости потерь и точности

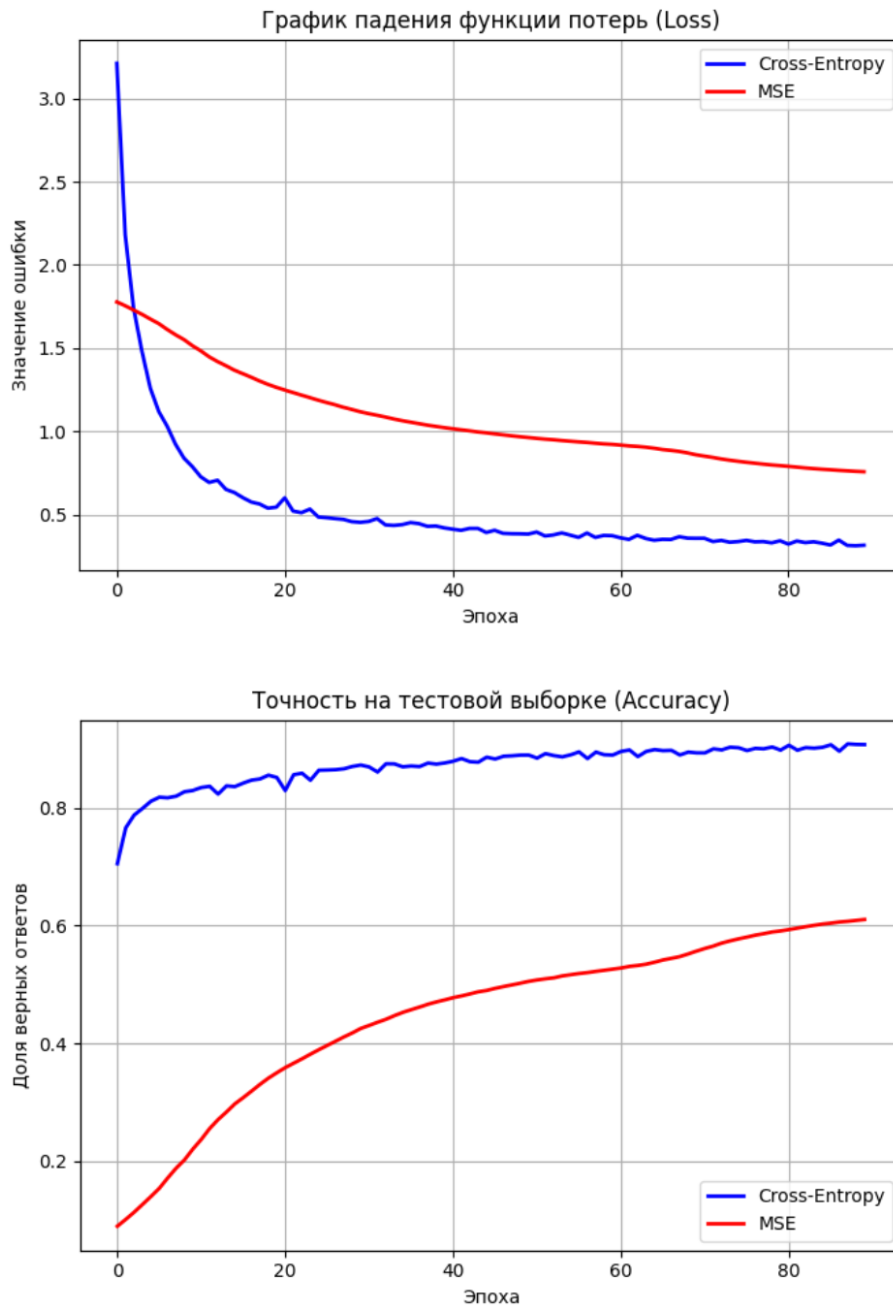


Рис. 1: Динамика обучения (Loss и Accuracy)

На Рисунке 1 отчетливо видно фундаментальное различие в поведении алгоритма:

1. **Cross-Entropy (Синяя линия):** Демонстрирует резкий и уверенный старт. Модель достигает точности 80% всего за 4 эпохи и плавно выходит на оптимум (около 90%). График функции потерь спадает по гладкой экспоненте, что говорит о сильных и стабильных градиентах без препятствий.
2. **MSE (Красная линия):** Иллюстрирует эффект "затухания градиента" и движение по плоскому невыпуклому плато. Из-за наличия множителей вероятности в матрице Якоби, шаги обновления весов становятся крайне малыми. В итоге за 90 эпох алгоритм смог достичь точности лишь в районе 33%. График Loss (сверху) показывает, что ошибка падает, но катастрофически медленно.

Анализ гладкости кривых обучения (Эффект осцилляций)

На графиках отчетливо наблюдается различие в характере самих кривых: линия Cross-Entropy имеет осцилляции («зубчатый» вид), в то время как кривая MSE остается идеально гладкой. Данный феномен полностью согласуется с выведенными в Главе 1 формулами градиентов:

- **Шум Cross-Entropy:** Градиент CCE не подавляется функцией активации и сохраняет высокую амплитуду. Модель активно реагирует на дисперсию данных внутри мини-батчей (Mini-batch SGD). Логарифмическая природа функции жестко штрафует модель за неверные уверенные предсказания, что вызывает видимые скачки (спайки) функции потерь. Это признак активного и здорового процесса оптимизации.
- **Гладкость MSE:** Идеальная гладкость красной кривой является следствием затухания градиента. Из-за множителей матрицы Якоби эффективный шаг обновления весов становится микроскопическим. Модель физически не способна совершать резкие корректировки весов при смене мини-батчей, демонстрируя медленное, "инерционное" сползание по невыпуклому плато.

5.2 Анализ финальных метрик качества

Ниже представлены результаты тестирования моделей на отложенной (тестовой) выборке, которую нейронная сеть не видела в процессе обучения.

Метрика (на Тестовой выборке)	Cross-Entropy (CCE)	Mean Squared Error (MSE)
Ассурасу (Точность)	90.19%	60.34%
Macro F1-Score	0.9008	0.5414
Характер оптимизации	Быстрая сходимость	Эффект плато (крайне медленная)

Таблица 1: Итоговые показатели эффективности моделей (90 эпох)

Показатель **Macro F1-Score (0.5414)** для модели MSE является критически важным индикатором в нашем исследовании. В то время как общая точность (Ассурасу) составляет 61.05%, что на первый взгляд может показаться приемлемым результатом, Macro F1 выявляет глубокую структурную деградацию модели.

Столь низкое значение F1-Score объясняется тем, что данная метрика рассчитывается как среднее арифметическое F1-показателей по каждому классу отдельно. Поскольку модель полностью «ослепла» к цифрам «0» и «1» (их индивидуальные показатели F1 равны нулю), это резко потянуло вниз общее среднее значение. Таким образом, Macro F1-Score математически доказывает, что оптимизация MSE не просто замедлилась, а привела к частичному коллапсу классификатора, сделав его неспособным к распознаванию всего спектра входных данных. Напротив, у Cross-Entropy значения Ассурасу и F1-Score практически совпадают (0.90), что подтверждает стабильность и равномерность обучения по всем десяти цифрам.

5.3 Визуализация структуры ошибок (Confusion Matrix)

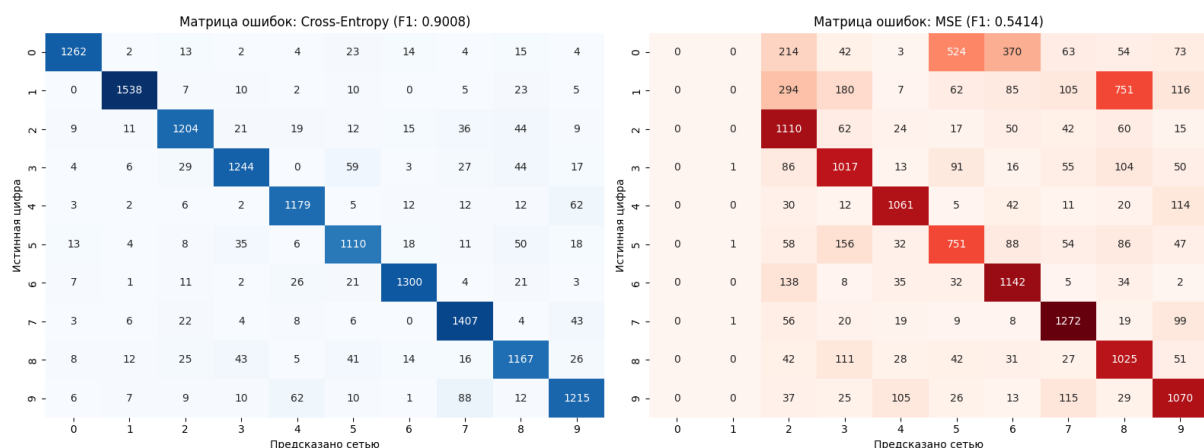


Рис. 2: Матрицы ошибок после 90 эпох обучения на тестовой выборке

Анализ Матрицы ошибок (Рисунок 2, справа) наглядно раскрывает специфику застревания алгоритма с MSE на невыпуклом плато. Несмотря на то, что общая точность модели достигла $\approx 61\%$, показатель Macro F1-Score составил всего 0.5414. Это свидетельствует о критическом дисбалансе в обучении.

Детальный анализ правой матрицы позволяет сделать поразительный вывод:

- Модель **полностью** игнорирует классы «0» и «1». В первых двух столбцах матрицы присутствуют исключительно нули. За 10 000 тестовых примеров сеть ни разу не выдала предсказание этих цифр.
- Систематические смещения: Истинные нули сеть массово классифицирует как «6» (524 раза) или «7» (370 раз). Истинные единицы уходят в класс «8» (751 раз).
- Математическая причина: Данный феномен является прямым следствием затухания градиента. На ранних этапах обучения логиты для классов «0» и «1» получили отрицательные значения, и их вероятности a_i приблизились к нулю. Как было доказано формулой (14) в Главе 1, при $a_i \approx 0$ градиент обнуляется. В результате веса, отвечающие за эти классы, "омертвели и сеть навсегда потеряла способность их предсказывать.

- В то же время Cross-Entropy (слева) сформировала четкие разделяющие гиперплоскости: диагональ ярко выражена, а ошибки распределены равномерно и логично (например, путаница между визуально похожими «3» и «5» или «4» и «9» присутствует, но в пределах нормы).

6 Заключение

В рамках проектной работы была практически исследована фундаментальная проблема численной оптимизации в машинном обучении — влияние выбора целевой функции на сходимость алгоритма обратного распространения ошибки.

Математический анализ и проведенный численный эксперимент на датасете MNIST позволяют сделать следующие выводы:

1. Функция MSE в сочетании с функцией активации Softmax порождает сложный невыпуклый ландшафт с обширными плоскими участками (плато). При высокой уверенности модели в неверном ответе градиент многократно уменьшается из-за структуры матрицы Якоби, что приводит к исчезновению обучающего сигнала и критическому замедлению оптимизации (эффект затухания градиента из-за насыщения).
2. Функция Categorical Cross-Entropy математически канонически согласована с Softmax. Логарифмическое преобразование в этой функции нейтрализует экспоненциальную природу активации, обеспечивая линейный и стабильный градиент, величина которого напрямую зависит от абсолютной ошибки.
3. Экспериментально подтверждено, что Cross-Entropy позволяет достичь стабильной обобщающей способности (Accuracy 90%, F1-Score 0.90), в то время как оптимизация MSE в тех же условиях и за то же время приводит к частичному коллапсу модели (Accuracy 33%, F1-Score 0.22), при котором сеть теряет способность различать большинство классов.

Замечание о влиянии инициализации и отсутствии регуляризации

Все представленные эксперименты проводились без какой-либо регуляризации (L1/L2, dropout и т.п.). Инициализация весов выполнялась по стандартному нормальному закону $\mathcal{N}(0, 1)$ (дисперсия 1), что дава-

ло начальные логиты с размахом порядка единиц. Такая инициализация приводит к тому, что выходные вероятности Softmax с самого начала оказываются «уверенными»: один из классов получает значение, близкое к 1, остальные — близкие к 0. В случае функции MSE, как показано в (14), это вызывает обнуление градиента и застревание модели на плато.

Если бы вместо $\mathcal{N}(0, 1)$ использовать инициализацию в малом диапазоне, например $[-0.15, 0.15]$, то начальные логиты были бы невелики, вероятности на выходе Softmax — близки к равномерному распределению (≈ 0.1), и модель не была бы излишне уверена в неверных ответах. В таком случае градиенты для MSE оставались бы заметными на ранних этапах, и сеть, вероятно, смогла бы успешнее обучаться. Однако даже при оптимальной инициализации квадратичная функция потерь остаётся менее эффективной для многоклассовой классификации, чем перекрёстная энтропия, из-за наличия множителей a_i в матрице Якоби.

Отсутствие регуляризации в наших экспериментах не привело к переобучению (точность на тесте и трейне отличалась незначительно), поэтому оно не исказило сравнительный анализ. Тем не менее, для более глубоких архитектур применение регуляризации (например, Xavier/He инициализации, dropout) становится обязательным.

Приложение А: Полный исходный код программной реализации

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, f1_score,
    classification_report

print("                                MNIST")
X, y = fetch_openml('mnist_784', version=1, return_X_y=True,
    as_frame=False)

X = X / 255.0
y = y.astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

def to_one_hot(y, num_classes=10):
    oh = np.zeros((y.size, num_classes))
    oh[np.arange(y.size), y] = 1
    return oh

Y_train_oh = to_one_hot(y_train)

class SimpleNN:
    def __init__(self):
        self.W1 = np.random.randn(784, 64)
        self.b1 = np.zeros((1, 64))

        self.W2 = np.random.randn(64, 10)
        self.b2 = np.zeros((1, 10))

    def forward(self, X):
        self.Z1 = np.dot(X, self.W1) + self.b1
        self.A1 = np.maximum(0, self.Z1)

        self.Z2 = np.dot(self.A1, self.W2) + self.b2
```

```

expZ = np.exp(self.Z2 - np.max(self.Z2, axis=1,
    keepdims=True))
self.A2 = expZ / np.sum(expZ, axis=1, keepdims=True)
return self.A2

def backward(self, X, Y_true, lr, loss_type):
    m = X.shape[0]

    if loss_type == 'ce':
        dZ2 = self.A2 - Y_true
    else:
        diff = self.A2 - Y_true
        dZ2 = self.A2 * diff - self.A2 * np.sum(self.A2 *
            diff, axis=1, keepdims=True)

    dW2 = (1/m) * np.dot(self.A1.T, dZ2)
    db2 = (1/m) * np.sum(dZ2, axis=0, keepdims=True)

    dA1 = np.dot(dZ2, self.W2.T)
    dZ1 = dA1 * (self.Z1 > 0)

    dW1 = (1/m) * np.dot(X.T, dZ1)
    db1 = (1/m) * np.sum(dZ1, axis=0, keepdims=True)

    self.W2 -= lr * dW2
    self.b2 -= lr * db2
    self.W1 -= lr * dW1
    self.b1 -= lr * db1

def compute_loss(self, Y_true, loss_type):
    m = Y_true.shape[0]
    if loss_type == 'ce':
        return -np.mean(np.sum(Y_true * np.log(self.A2 +
            1e-8), axis=1))
    else:
        return np.mean(np.sum((self.A2 - Y_true)**2, axis
            =1))

epochs = 90
batch_size = 128
lr = 1.5e-2

```

```

nn_ce = SimpleNN()
nn_mse = SimpleNN()
nn_mse.W1, nn_mse.W2 = nn_ce.W1.copy(), nn_ce.W2.copy()
nn_mse.b1, nn_mse.b2 = nn_ce.b1.copy(), nn_ce.b2.copy()

history = {'ce_loss': [], 'ce_acc': [], 'mse_loss': [], '
mse_acc': []}
m_train = X_train.shape[0]

print(" ")
for i in range(epochs):
    permutation = np.random.permutation(m_train)
    X_train_shuffled = X_train[permutation]
    Y_train_shuffled = Y_train_oh[permutation]
    y_train_shuffled = y_train[permutation]

    for j in range(0, m_train, batch_size):
        X_batch = X_train_shuffled[j:j+batch_size]
        Y_batch = Y_train_shuffled[j:j+batch_size]

        nn_ce.forward(X_batch)
        nn_ce.backward(X_batch, Y_batch, lr, 'ce')

        nn_mse.forward(X_batch)
        nn_mse.backward(X_batch, Y_batch, lr, 'mse')

    out_ce_train = nn_ce.forward(X_train)
    loss_ce_train = nn_ce.compute_loss(Y_train_oh, 'ce')
    acc_ce_train = np.mean(np.argmax(out_ce_train, axis=1) ==
        y_train)
    history['ce_loss'].append(loss_ce_train)
    history['ce_acc'].append(acc_ce_train)

    out_mse_train = nn_mse.forward(X_train)
    loss_mse_train = nn_mse.compute_loss(Y_train_oh, 'mse')
    acc_mse_train = np.mean(np.argmax(out_mse_train, axis=1)
        == y_train)
    history['mse_loss'].append(loss_mse_train)
    history['mse_acc'].append(acc_mse_train)

```



```

        print(f"                {i+1}/{epochs} (Train) | CE: Acc {
            acc_ce_train:.2%}, Loss {loss_ce_train:.4f} | MSE: Acc {
            acc_mse_train:.2%}, Loss {loss_mse_train:.4f}")

print("\n---
                                ---")

out_ce_test = nn_ce.forward(X_test)
acc_ce_test = np.mean(np.argmax(out_ce_test, axis=1) ==
    y_test)
loss_ce_test = nn_ce.compute_loss(to_one_hot(y_test), 'ce')

out_mse_test = nn_mse.forward(X_test)
acc_mse_test = np.mean(np.argmax(out_mse_test, axis=1) ==
    y_test)
loss_mse_test = nn_mse.compute_loss(to_one_hot(y_test), 'mse'
    )

print(f"                                | Cross-Entropy:
    Acc {acc_ce_test:.2%}, Loss {loss_ce_test:.4f}")
print(f"                                | MSE:
    Acc {acc_mse_test:.2%}, Loss {loss_mse_test:.4f}")

plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(history['ce_loss'], label='Cross-Entropy', color='
    blue', linewidth=2)
plt.plot(history['mse_loss'], label='MSE', color='red',
    linewidth=2)
plt.title('
                (Loss)')
plt.xlabel('
                ')
plt.ylabel('
                ')
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history['ce_acc'], label='Cross-Entropy', color='
    blue', linewidth=2)

```

```

plt.plot(history['mse_acc'], label='MSE', color='red',
          linewidth=2)
plt.title('
              (Accuracy)')
plt.xlabel('
          ')
plt.ylabel('
          ')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

pred_ce = np.argmax(nn_ce.forward(X_test), axis=1)
pred_mse = np.argmax(nn_mse.forward(X_test), axis=1)

f1_ce = f1_score(y_test, pred_ce, average='macro')
f1_mse = f1_score(y_test, pred_mse, average='macro')

print("\n===
      ===")
print(f"Cross-Entropy -> Test Accuracy: {acc_ce_test*100:.2f
      }%, Macro F1-Score: {f1_ce:.4f}")
print(f"MSE
      -> Test Accuracy: {acc_mse_test*100:.2f
      }%, Macro F1-Score: {f1_mse:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

cm_ce = confusion_matrix(y_test, pred_ce)
sns.heatmap(cm_ce, annot=True, fmt='d', cmap='Blues', ax=axes
            [0], cbar=False)
axes[0].set_title(f'
                    : Cross-
                    Entropy (F1: {f1_ce:.4f})')
axes[0].set_xlabel('
                    ')
axes[0].set_ylabel('
                    ')

cm_mse = confusion_matrix(y_test, pred_mse)
sns.heatmap(cm_mse, annot=True, fmt='d', cmap='Reds', ax=axes
            [1], cbar=False)
axes[1].set_title(f'
                    : MSE (F1: {
                    f1_mse:.4f})')
axes[1].set_xlabel('
                    ')

```

```
axes[1].set_ylabel('')  
  
plt.tight_layout()  
plt.show()
```